

DRD · SPARK HACK TORONTO · v2.0

CivicVox-Omni

Design & Architecture Reference Document — 5-Person Team Edition

Version: 2.0 — DRAFT **Date:** May 29, 2026 **Supersedes:** DRD v1.0 (3-person)

1. What Changed in v2.0

DRD v1.0 assumed a 3-person team split across Infrastructure, Agent/Data, and Frontend. With 5 people the workload can be divided more finely, eliminating the bottlenecks that caused the most risk in the original plan — specifically, the single person responsible for both the FastAPI gateway and the local AI inference setup, and the single person responsible for both LangGraph orchestration and all geospatial data work.

Area	v1.0 (3 people)	v2.0 (5 people)
Inference & GPU setup	Person 1 (alongside API gateway)	Person 1 — dedicated
API gateway + STT/TTS	Person 1 (alongside GPU)	Person 2 — dedicated
Agent orchestration (LangGraph)	Person 2 (alongside GIS data)	Person 3 — dedicated
GIS data + Open Data RAG	Person 2 (alongside agents)	Person 4 — dedicated
Frontend dashboard	Person 3 — sole owner	Person 5 — dedicated
Integration test	Hour 12	Hour 10 (earlier — more buffer)

2. Team Roles & Ownership

Each person owns one layer end-to-end. No shared files within a layer; cross-layer communication is via the JSON contracts in Section 7.

Person	Title	Primary deliverable	Interface output
Person 1	GPU & Inference Engineer	NVIDIA NIM / vLLM running on GB10; Whisper + Kokoro loaded	http://localhost:8000/v1 (OpenAI-compatible endpoint)
Person 2	API Gateway Engineer	FastAPI server; STT/TTS pipelines; WebSocket telemetry	http://0.0.0.0:8080 endpoints (see Section 4)
Person 3	Agent Orchestration Engineer	LangGraph StateGraph; all 4 agent nodes; system prompts	civic_vox_engine.invoke() importable by Person 2
Person 4	Data & GIS Engineer	Toronto Open Data download; GeoPandas loader; spatial query fns	toronto_loader.py importable by Person 3
Person 5	Frontend Engineer	Next.js dashboard; Mapbox map; agent telemetry panel; camera capture	Runs at http://localhost:3000

3. System Architecture Overview

CivicVox-Omni is composed of five independently buildable modules mapped 1-to-1 to the five team members. All modules run on the ASUS GX10 (NVIDIA GB10 Grace Blackwell Superchip) local network — zero external API calls during operation.

Module	Tech stack	Owner
Local AI inference server	NVIDIA NIM / vLLM / Ollama, TensorRT-LLM, CUDA drivers	Person 1
API gateway + speech pipelines	FastAPI, Uvicorn, Faster-Whisper, Kokoro-82M ONNX	Person 2
Agent orchestration engine	LangGraph, LangChain, LangChain-OpenAI, Python 3.11	Person 3
GIS data module	GeoPandas, Shapely, PyProj, SciPy R-tree, Toronto Open Data	Person 4
Frontend dashboard	Next.js 14, TypeScript, Tailwind CSS, Framer Motion, Mapbox GL JS	Person 5

4. Person 1 — GPU & Inference Engineer

Person 1's sole responsibility is getting open-weight models running on the GB10 and exposing a stable local inference endpoint. Everything else depends on this being done in the first 3 hours.

4.1 Hour-by-hour plan

Time	Task	Done when...
0 – 1 h	SSH into GX10; verify CUDA drivers and GPU visibility (nvidia-smi)	nvidia-smi shows the GB10 with correct VRAM
1 – 2 h	Pull NVIDIA NIM container for Llama-3.2-11B-Vision-Instruct OR install vLLM via pip	Container starts without errors
2 – 3 h	Run a test curl to POST /v1/chat/completions with a text-only prompt; verify JSON response	curl returns a valid completion JSON
3 – 5 h	Test a multimodal (image + text) request with a base64 JPEG; confirm vision inference works	Model returns structured JSON from an image prompt
5 – 8 h	Load Faster-Whisper Large-v3 on CUDA; write transcribe_audio(pcm_bytes) test script	10-second PCM clip transcribed in < 300 ms

Time	Task	Done when...
8 – 12 h	Load Kokoro-82M ONNX on CPU; write <code>synthesize_speech(text)</code> test script; share both with Person 2	WAV bytes returned for a test sentence in < 100 ms

4.2 Model configuration

Model	Task	Backend	Quant	VRAM	Latency target
Llama-3.2-11B-Vision-Instruct	VLM — vision + text	NIM / vLLM	INT4 / FP8	~8 GB	< 60 ms/token
Whisper Large-v3	Speech-to-text	faster-whisper (CUDA)	FP16	~3 GB	< 300 ms / 10 s
Kokoro-82M	Text-to-speech	kokoro-onnx (CPU)	FP32 ONNX	< 1 GB (CPU)	< 100 ms / sentence

4.3 Inference server options

Option	Setup command	Notes
NVIDIA NIM (preferred)	<code>docker run --gpus all nvcr.io/nim/meta/llama-3.2-11b-vision-instruct</code>	Best GB10 utilisation; requires Docker + NGC credentials
vLLM (fallback)	<code>pip install vllm && python -m vllm.entrypoints.openai.api_server --model meta-llama/Llama-3.2-11B-Vision-Instruct</code>	No Docker; easiest install
Ollama (dev testing)	<code>ollama serve && ollama pull llama3.2-vision</code>	Lowest throughput; use only for initial smoke test

Environment variable

Set `LOCAL_LLM_URL=http://localhost:8000/v1` — this is the only config Person 2 and Person 3 need from Person 1.

5. Person 2 — API Gateway Engineer

Person 2 builds the FastAPI server that acts as the sole network entry point. It receives media payloads from the frontend, runs speech processing, delegates to the agent engine, and streams results back. Person 2 does not write any agent logic — they call Person 3's engine as a black box.

5.1 Hour-by-hour plan

Time	Task	Done when...
0 – 2 h	Scaffold FastAPI app; define Pydantic request/response models for all 4 endpoints; add health check	GET /api/health returns { status: ok }
2 – 4 h	Integrate Person 1's transcribe_audio() into POST /api/incident handler using stub agent call	STT result printed to console for a test PCM upload
4 – 7 h	Integrate Person 3's civic_vox_engine.invoke() into the route handler; wire AgentState construction	Full mock state flows through engine, JSON response returned
7 – 9 h	Build WebSocket /ws/stream endpoint; emit agent node completion events as SSE-style JSON frames	Dashboard receives { node, status, timestamp } events live
9 – 12 h	Integrate Person 1's synthesize_speech(); wire POST /api/synthesize; handle WAV streaming response	Browser plays back synthesized audio from a test sentence

5.2 Endpoint specifications

Method	Endpoint	Request body	Response
POST	/api/incident	{ transcript: str, frame_b64: str, gps: {lat, lng} }	{ report, vision, spatial, urgency }
WS	/ws/stream	Binary audio + JSON metadata frames	{ node, status, timestamp } events
POST	/api/synthesize	{ text: str }	WAV bytes (audio/wav)
GET	/api/health	—	{ status, models_loaded }

5.3 Key implementation notes

- Bind to 0.0.0.0:8080 — the GB10's LAN IP must be reachable from Person 5's frontend and from mobile demo devices.
- Set CORS origins to ["*"] for hackathon (all devices on the same local Wi-Fi).
- Use asyncio + FastAPI async def endpoints; call civic_vox_engine.ainvoke() for non-blocking agent execution.
- Import toronto_loader.py at server startup so GeoDataFrames are warm before the first request.
- AgentState construction lives in server.py — Person 2 assembles the dict and passes it to Person 3's engine.

Mock stub pattern

Person 2 should implement a `MOCK_MODE` env flag. When set, the `/api/incident` endpoint returns hardcoded JSON instead of calling the agent engine. This lets Person 5 build the entire frontend against a stable response before the agents are ready.

6. Person 3 — Agent Orchestration Engineer

Person 3 owns the LangGraph state machine — the cognitive core of CivicVox-Omni. Their module (backend/agents/civic_vox_graph.py) is imported by Person 2 and calls Person 4's spatial query functions. Person 3 does not write any HTTP server code or frontend code.

6.1 Hour-by-hour plan

Time	Task	Done when...
0 – 2 h	Define AgentState TypedDict; scaffold 4 node function stubs returning mock dicts; compile graph	civic_vox_engine.invoke(mock_state) runs without error
2 – 5 h	Implement Orchestrator node: send transcript to VLM; parse routing JSON safely	Routing JSON parsed; next_step set correctly
5 – 8 h	Implement Vision node: multimodal VLM call with base64 frame; parse hazard JSON output	vision_analysis dict populated from a real image prompt
8 – 10 h	Implement Localizer node: wire Person 4's get_closest_hydrants() and get_building_specs()	spatial_data_results dict populated with real GIS data
10 – 12 h	Implement Compiler node: synthesis prompt → dispatch report text; run end-to-end test	final_dispatch_report string non-empty for a real incident payload

6.2 AgentState schema

TypedDict fields

messages: list[BaseMessage] · video_frames_base64: list[str] · gps_coordinates: dict · vision_analysis: dict · spatial_data_results: dict · next_step: str · final_dispatch_report: str

6.3 Node specifications

Node	Reads from state	Writes to state	LLM calls
Orchestrator	messages[-1] (transcript)	next_step, urgency_level	1 — text routing prompt
Vision Agent	video_frames_base64[-1], messages	vision_analysis, next_step	1 — multimodal (text + image_url)
Localizer Agent	gps_coordinates, vision_analysis	spatial_data_results, next_step	0 — GeoPandas only (Person 4's functions)
Report Compiler	messages, vision_analysis, spatial_data_results	final_dispatch_report, next_step	1 — synthesis prompt

6.4 Graph routing table

From node	Condition	To node
orchestrator	next_step == 'vision'	vision
orchestrator	next_step == 'localizer' (no camera / audio-only incident)	localizer
vision	always	localizer
localizer	always	compiler
compiler	always	END

6.5 System prompts

Orchestrator prompt

```
You are the emergency coordinator for CivicVox-Omni, Toronto.
Analyse this citizen report: "{transcript}"
Output ONLY valid JSON (no markdown fences):
{ "requires_vision": bool, "urgency_level": "LOW"|"HIGH"|"CRITICAL" }
```

Vision Agent prompt

```
Analyse this emergency video frame. Output ONLY valid JSON (no markdown fences):
{ "hazard_type": string, "severity_scale": integer 1-10,
  "water_depth_m": float or null, "structural_risk": boolean,
  "location_cues": string }
```

Report Compiler prompt

```
You are Toronto Emergency Command. Given the inputs below, produce a
concise priority-coded dispatch protocol covering:
1. Threat classification and recommended perimeter radius
2. Infrastructure access nodes with exact distances
3. Building vulnerabilities
4. Recommended crew type and count
```

JSON safety

Open-weight VLMs often wrap JSON in ``json ... `` fences. Always strip these before calling `json.loads()`. Use: `import re; clean = re.sub(r'``[w]*\n?``', "", raw).strip()`

6.6 Pip dependencies (Person 3)

```
pip install langgraph langchain-core langchain-openai
```

7. Person 4 — Data & GIS Engineer

Person 4 downloads all Toronto Open Data, builds the in-memory spatial index, and exposes two clean query functions imported by Person 3. This module has zero LLM dependency — it is pure Python geospatial work.

7.1 Hour-by-hour plan

Time	Task	Done when...
0 – 2 h	Download all 4 Open Data files to /backend/data/ using the CKAN API or direct download links	All GeoJSON/CSV files present on disk
2 – 4 h	Write toronto_loader.py: load all GeoDataFrames; project to EPSG:2958; build R-tree index on hydrants	Loader runs without error; hydrants GDF has 'distance' column method ready
4 – 6 h	Implement get_closest_hydrants(lat, lng, n=3) -> list[dict]	Returns 3 nearest hydrants with id, distance_meters, status in < 20 ms
6 – 8 h	Implement get_building_specs(lat, lng) -> dict: reverse-geocode to nearest RentSafeTO record	Returns floors, contact, last_inspection_date for a test coordinate
8 – 10 h	Implement get_nearest_road(lat, lng) -> dict using Street Centrelines GDF	Returns road name and distance for a test coordinate
10 – 12 h	Write unit tests for all 3 functions; share module with Person 3 for integration	All 3 functions return expected dicts; no errors on 5 test coordinates

7.2 Toronto Open Data sources

Dataset	Format	Portal path	In-memory object
Fire Hydrant locations	GeoJSON	open.toronto.ca/dataset/fire-hydrants	GeoDataFrame (EPSG:2958) + R-tree index
RentSafeTO apartment evaluations	CSV	open.toronto.ca/dataset/apartment-building-evaluation	GeoDataFrame (geocoded, EPSG:2958)
Street Centrelines	GeoJSON	open.toronto.ca/dataset/toronto-centreline-tcl	GeoDataFrame (EPSG:2958)
311 Service Requests	CSV	open.toronto.ca/dataset/311-service-requests-customer-initiated	Pandas DataFrame (filtered by incident type)

7.3 Public interface contract

```
# backend/data/toronto_loader.py

def get_closest_hydrants(lat: float, lng: float, n: int = 3) -> list[dict]:
    # Returns: [{ 'id': int, 'distance_meters': float, 'status': str }, ...]

def get_building_specs(lat: float, lng: float) -> dict:
```



```
# Returns: { 'address': str, 'floors': int, 'contact': str, 'last_inspection': str }

def get_nearest_road(lat: float, lng: float) -> dict:
    # Returns: { 'road_name': str, 'distance_meters': float }
```

Performance note

Pre-load all GeoDataFrames and the R-tree index into module-level globals at import time — not inside each function call. This eliminates disk I/O from the critical path and keeps spatial queries under 20 ms.

7.4 Pip dependencies (Person 4)

```
pip install geopandas shapely pyproj scipy pandas requests
```

8. Person 5 — Frontend Engineer

Person 5 owns the Next.js dashboard — the visual interface judges will watch during the demo. They work entirely in the /frontend directory and never touch Python. From Hour 0 they build against Person 2's mock JSON stubs; they switch to the live backend at Hour 10 integration.

8.1 Hour-by-hour plan

Time	Task	Done when...
0 – 2 h	Scaffold Next.js 14 app with Tailwind; set up .env.local with API_URL and Mapbox token; create 3-column grid layout	npx next dev runs; dark background visible
2 – 5 h	Build <MapView /> with Mapbox GL dark-matter tiles; add static incident pin and hydrant markers from mock JSON	Map renders with 3 coloured markers
5 – 7 h	Build <AgentPipeline /> with 4 <AgentCard /> nodes showing idle/active/complete states with Framer Motion animations	Cards animate through states when a button is clicked
7 – 9 h	Build <CameraCapture /> using MediaDevices API: capture JPEG every 2 s, POST to Person 2's /api/incident (mock mode)	Console logs show base64 frames being captured
9 – 10 h	Build <DispatchReport /> with token-by-token typewriter render of the report string; add TTS audio playback widget	Mock report text streams character-by-character
10 – 12 h	Switch API_URL to live GB10 LAN IP; wire WebSocket /ws/stream for live agent telemetry; demo rehearsal	Full end-to-end loop works on real video input

8.2 Component inventory

Component	File	Responsibility
MapView	components/MapView.tsx	Mapbox GL JS dark map; incident pin; hydrant/building asset overlays; alert radius circle
AgentPipeline	components/AgentPipeline.tsx	4-node pipeline visualisation; pulsing Framer Motion ring on active node; green check on complete
AgentCard	components/AgentCard.tsx	Individual node card: label, status badge, timestamp
CameraCapture	components/CameraCapture.tsx	MediaDevices getUserMedia; canvas snapshot loop every 2 s; audio MediaRecorder; sends to API
DispatchReport	components/DispatchReport.tsx	Typewriter text render; copy-to-clipboard; audio playback via /api/synthesize
IncidentBadge	components/IncidentBadge.tsx	Colour-coded urgency badge (LOW/HIGH/CRITICAL) shown top-right of map

8.3 Environment variables

Variable	Default	Set by
NEXT_PUBLIC_API_URL	http://localhost:8080	Person 5 — update to GB10 LAN IP before demo
NEXT_PUBLIC_WS_URL	ws://localhost:8080/ws/stream	Person 5 — update to GB10 LAN IP before demo
NEXT_PUBLIC_MAPBOX_TOKEN	pk.eyJ1Ijoi...	Person 5 — free public token from mapbox.com

8.4 Npm dependencies

```
npm install mapbox-gl react-map-gl framer-motion @tailwindcss/typography
```

9. Integration Protocol

With 5 people, integration risk increases. The following handshake protocol keeps everyone unblocked and surfaces problems early.

9.1 Interface freeze — Hour 3

By Hour 3, all five people must agree on the JSON contracts defined in Section 10. No interface may change after this point without a verbal team check-in. Person 2 publishes the `MOCK_MODE` stub immediately after the freeze so Person 5 can start building against real response shapes.

9.2 Component handshake milestones

Hour	Handshake	Parties	How to verify
3	Inference endpoint live	P1 → P2, P3	curl POST to localhost:8000/v1/chat/completions returns completion JSON
5	STT pipeline live	P1 → P2	Person 2 calls <code>transcribe_audio()</code> on a test WAV file; transcript printed
7	GIS functions live	P4 → P3	Person 3 imports <code>toronto_loader</code> ; calls <code>get_closest_hydrants(43.66, -79.39, 3)</code> ; dict returned
9	Agent engine live	P3 → P2	Person 2 calls <code>civic_vox_engine.invoke(mock_state)</code> ; <code>final_dispatch_report</code> non-empty
10	Full integration test	All	Person 5 submits a real camera frame; map updates with live data; agent cards animate
12	Demo freeze	All	No code changes to core paths; only UI polish and demo script refinement allowed

9.3 Parallel work rules

- No shared Python files between people. Each person writes to their own module; imports are one-directional (P2 imports P3; P3 imports P4; no cycles).
- All cross-team data is passed as plain Python dicts matching the contracts in Section 10 — no shared class instances.
- If a dependency is not ready, use the mock stub pattern: return the hardcoded example JSON from Section 10 until the real implementation arrives.
- Person 5 never calls Person 3 or Person 4 directly — all frontend requests go through Person 2's API.

10. Interface Contracts (Source of Truth)

All five team members must agree on these schemas at Hour 3. They are the immutable API between all modules.

10.1 Frontend → Gateway (POST /api/incident)

```
{
  "transcript": "There is flooding in my basement",
  "frame_b64": "<JPEG encoded as base64 string>",
  "gps": { "lat": 43.6629, "lng": -79.3957 }
```

```
}
```

10.2 Gateway → Frontend (response)

```
{
  "urgency": "CRITICAL",
  "report": "THREAT: Flash flooding - Category 3 ...",
  "vision": { "hazard_type": "Flooding", "severity_scale": 7,
              "water_depth_m": 0.4, "structural_risk": false,
              "location_cues": "Brick residential building, north-facing" },
  "spatial": {
    "closest_hydrant": { "id": 1042, "distance_meters": 38.5, "status": "Operational" },
    "building_specs": { "address": "123 Example St", "floors": 12,
                        "contact": "Toronto Housing Corp", "last_inspection": "2025-11-14" },
    "nearest_road": { "road_name": "Spadina Ave", "distance_meters": 12.0 }
  }
}
```

10.3 WebSocket telemetry event

```
{ "node": "vision", "status": "active", "timestamp": "2026-05-29T21:05:33.412Z" }
{ "node": "vision", "status": "complete", "timestamp": "2026-05-29T21:05:33.890Z" }
```

10.4 Gateway → Agent engine (AgentState dict)

```
{
  "messages": [HumanMessage(content=transcript)],
  "video_frames_base64": [frame_b64],
  "gps_coordinates": { "lat": 43.6629, "lng": -79.3957 },
  "vision_analysis": {},
  "spatial_data_results": {},
  "next_step": "orchestrator",
  "final_dispatch_report": ""
}
```

10.5 GIS module functions (Person 4 → Person 3)

```
get_closest_hydrants(43.6629, -79.3957, n=3)
# Returns: [{ 'id': 1042, 'distance_meters': 38.5, 'status': 'Operational' }, ...]

get_building_specs(43.6629, -79.3957)
# Returns: { 'address': '123 Example St', 'floors': 12,
#           'contact': 'Toronto Housing Corp', 'last_inspection': '2025-11-14' }
```

11. Hackathon Timeline

All five tracks run in parallel. The critical path is: Person 1 inference live (Hour 3) → Person 4 GIS ready (Hour 7) → Person 3 agents integrated (Hour 9) → full integration test (Hour 10) → polish (Hour 12+).

Hour	P1 — Inference	P2 — Gateway	P3 — Agents	P4 — GIS Data	P5 — Frontend
0–2	CUDA verify; pull NIM image	Scaffold FastAPI; health endpoint	AgentState + stub nodes; graph compiles	Download all 4 Open Data files	Next.js scaffold; layout grid; Mapbox token
2–4	Test text inference via curl	Wire STT pipeline (P1 dep)	Orchestrator node + routing	Build GeoDataFrame loader; EPSG:2958	MapView with static markers
4–6	Test multimodal (image) inference	Stub agent call in route handler	Vision node + multimodal call	get_closest_hydrants() function	AgentPipeline + AgentCard animations
6–8	Load Whisper; latency test	WebSocket telemetry endpoint	Localizer node (P4 dep)	get_building_specs() + get_nearest_road()	CameraCapture component
8–10	Load Kokoro TTS; share fns with P2	TTS endpoint; end-to-end gateway test	Compiler node; full graph end-to-end test	Unit tests for all 3 GIS functions	DispatchReport typewriter + audio widget
10–12	GPU perf tuning; TensorRT flags	Live integration; fix edge cases	Prompt tuning; JSON safety hardening	Performance profiling; R-tree verification	Switch to live backend; WebSocket telemetry live
12+	Monitor inference speed for pitch	API stability; demo script	System prompt polish	Extra: 311 data filter for context	UI animations; demo rehearsal

12. Risk Register (5-Person Edition)

Risk	Likely.	Impact	Owner	Mitigation
NIM container fails to start on GB10	Med	Critical	P1	Pre-test vLLM as fallback the week before; have Ollama as last resort
VLM VRAM overflow with INT8	Med	High	P1	Pre-quantise to INT4 with TensorRT-LLM; drop to Llama-3.2-3B if needed
GIS files too large to download on venue Wi-Fi	High	High	P4	Download all datasets before the event; commit to repo or USB drive

Risk	Likely.	Impact	Owner	Mitigation
Interface contract breaks mid-event	Med	High	P2	Freeze contracts at Hour 3; use MOCK_MODE so P5 never blocks on backend
Frontend can't reach GB10 on venue network	Med	High	P5	Test local Wi-Fi hotspot fallback; confirm GB10 LAN IP before demo
Open-weight VLM produces non-JSON output	High	Med	P3	Regex strip + try/except in every agent node; fallback to raw text if parse fails
WebSocket drops during demo	Low	Med	P2	Fallback to HTTP polling mode (2 s interval) — same UX, no real-time telemetry
Team integration test fails at Hour 10	Med	High	All	Each person has a mock stub ready; can demo individual layers if full chain fails

13. Repository Structure

Monorepo layout

```
/civicvox-omni /backend server.py # Person 2 — FastAPI entry point /agents
civic_vox_graph.py # Person 3 — LangGraph state machine /data toronto_loader.py #
Person 4 — GeoDataFrame loader + spatial fns download_data.py # Person 4 — one-shot
Open Data downloader *.geojson / *.csv # Person 4 — pre-downloaded Open Data files
inference/ whisper_pipeline.py # Person 1 — STT utility kokoro_pipeline.py # Person 1
— TTS utility /frontend app/ page.tsx # Person 5 — main dashboard page
components/ MapView.tsx # Person 5 AgentPipeline.tsx # Person 5
AgentCard.tsx # Person 5 CameraCapture.tsx # Person 5 DispatchReport.tsx #
Person 5 IncidentBadge.tsx # Person 5 .env.local # Person 5 — env vars (never
commit) README.md .env.example # shared env var template
```

14. Demo Script & Pitch Talking Points

Moment	Who drives	What to show	What to say
Open with the problem	P5	Static slide: city emergency, cloud fails	"During a major emergency, cloud latency is a liability. We eliminate it entirely."
Live camera demo	P5 on phone	Phone camera pointed at staged flooding	"Any citizen. No app install. Just a browser on the local network."
Vision fires	P5 watches dashboard	Vision Agent card pulses → completes; map pin drops	"The GB10 just analysed that frame locally. No data left this room."

Moment	Who drives	What to show	What to say
Spatial query	P5 + P4 narrates	Hydrant markers appear on map	"Three nearest operational hydrants from Toronto Open Data — returned in under 20 ms."
Dispatch report	P5 reads panel	Typewriter text streams in report panel	"Military-grade dispatch protocol, at edge speed."
Hardware close	P1 narrates	Show nvidia-smi / GPU utilisation dashboard	"This is the NVIDIA GB10 doing what no cloud can: real-time multi-modal AI, fully sovereign."